Assignment 3 — Bridge Pattern

Astana IT University · ShP-2216 Software Design Patterns · 2026–2027 · Individual assignment
Student: 李易鑫 (Yixin Li) · **Group:** SE-2526 · **Topic:** A — Drawing / Shapes
Repository: https://github.com/CCE-Li/AITU-SDP-Assignment3 · **Base commit:** 5dae42bf5297e12ebb6cd27da719863c718033e5

1. Two independent dimensions — and why Bridge fits

The application varies along two dimensions that are independent of each other.

- **Abstraction — "what is drawn":** `Circle` (A1, radius = 2) and `Square` (A2, side = 3). Each abstraction has an ID and keeps its own domain data.
- **Implementation — "how it is drawn":** `VectorRenderer` (I1), `RasterRenderer` (I2), and in the extension `AsciiRenderer` (I3). Each renderer turns a shape description into its own text result.

Multiplied into one hierarchy, every combination would need its own class (2 shapes × 3 renderers = 6) and every new renderer would touch every shape. Bridge keeps the two hierarchies separate and links them by composition: the abstract base `Shape` stores an interface-typed reference (`Renderer renderer`) injected through its constructor, and it delegates the low-level operation in `execute()`. Only 2 + 3 = 5 classes are needed, combinations are formed at runtime through `setImplementation(...)`, and the extension (I3) was added without editing a single existing shape or renderer — verified by `extension.diff`.

2. UML class diagram

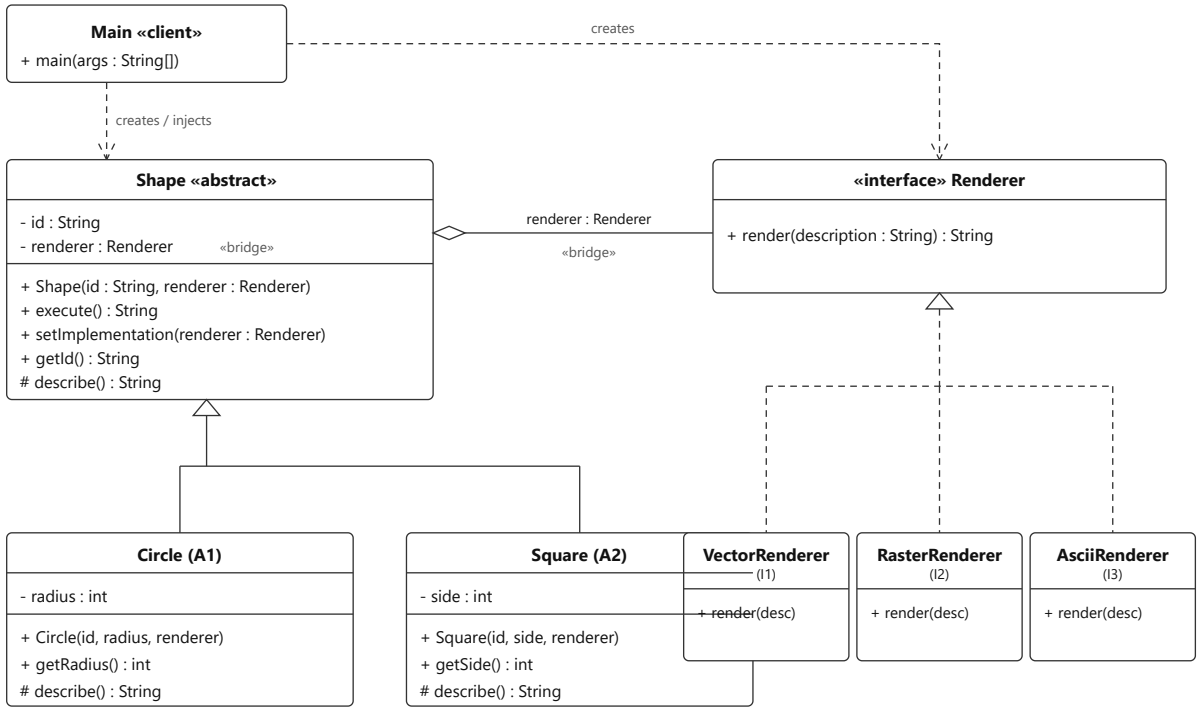


Figure 1 — Final code (Topic A). Hollow triangle: generalization / interface realization (dashed); hollow diamond: aggregation — the private `renderer` bridge field; dashed open arrows: client dependency. `Main` creates shapes and injects renderers; `Shape` only talks to the `Renderer` interface.

3. Five annotated excerpts from the submitted code

3.1 Separation of responsibilities

```
// src/Circle.java — domain data and its textual form
private final int radius;

@Override
protected String describe() {
    return "circle radius=" + radius;
}

// src/VectorRenderer.java — rendering technology, no shape knowledge
@Override
public String render(String description) {
    return "VECTOR " + description;
}
```

Benefit: shapes know nothing about rendering, and renderers know nothing about shapes, so either dimension can change without touching the other.

3.2 Meaningful names

```
// src/Renderer.java — the Implementor role of the pattern
public interface Renderer {
    String render(String description);
}

// src/Shape.java — the name says exactly what the method does
public void setImplementation(Renderer renderer) {
    this.renderer = renderer;
}
```

Benefit: the code reads in the pattern's own vocabulary (`Renderer`, `VectorRenderer`, `execute`, `setImplementation`); the same names are used in the README and this report, so the two hierarchies are easy to find during review.

3.3 Small, focused methods

```
// src/Main.java — one ~10-line method is the whole checking workflow
private static void verify(String testId, String participants, boolean domainOk, String actual, String
expected) {
    total++;
    boolean pass = domainOk && expected.equals(actual);
    if (pass) {
        passed++;
    }
    System.out.println(testId + (pass ? " PASS" : " FAIL") + " | " + participants
            + " | result=" + actual + (pass ? "" : " | expected=" + expected + " | domainOk=" +
domainOk));
}
```

Benefit: every T1–T7 check is a single call to `verify(...)`; there is exactly one place where PASS/FAIL is computed, and it is calculated from the real result — the summary `SUMMARY: 7/7 PASS` is never hard-coded.

3.4 No duplicated workflow (no combination subclasses)

```
// src/Shape.java — the only decision point between abstraction and implementation
public String execute() {
    return renderer.render(describe());
}
```

Benefit: all seven checks flow through this single line; there is no `if`/`switch` on renderer types and no `CircleWithVector`-style class. Adding I3 required only a new renderer class, so no workflow or formatting logic is duplicated per combination.

3.5 Encapsulation of state

```
// src/Shape.java
private final String id;            // identity, exposed read-only via getId()
private Renderer renderer;          // private bridge state

public void setImplementation(Renderer renderer) {   // the single mutation point
    this.renderer = renderer;
}
```

Benefit: the abstraction's ID and domain data cannot be corrupted from outside; T5 confirms that after the runtime switch the reference is unchanged (`sameObject=true`) and the state is unchanged (`stateUnchanged=true`) while only the result changes.

4. Extension with I3 · trade-off · Bridge vs Adapter

Extension. I3 was added as an independent extension: `src/AsciiRenderer.java` is one new class that implements `Renderer`. No abstraction class, no interface signature, and neither I1 nor I2 changed; inside `src/` only `Main.java` gained the T6/T7 checks (plus `sources.txt` and the README). `extension.diff` — the diff from the base commit, limited to `src/` — shows exactly that: one new file and one updated file. This is the practical proof that implementations are pluggable through the bridge.

Trade-off. Bridge costs indirection: every execution crosses the interface boundary (`execute() → renderer.render()`), the implementor operation is an API contract both hierarchies must respect (here: one `render(String)`), and there are more, smaller types to read than in a two-class program. That price pays off only when both dimensions truly vary; if shapes and renderers always changed together, plain subclassing would be simpler.

Bridge vs Adapter. Both patterns use composition and wrapping, but their intent differs. Adapter makes an existing, incompatible interface usable by a client — a conversion applied *after the fact* to one class that already exists (intent: compatibility). Bridge is a design decision made *before* the combinations multiply: it separates an abstraction from its implementation so both can vary independently (intent: decoupling). For example, if a legacy `LegacyCanvas.print(String)` had to satisfy our `Renderer`, the wrapper would be an Adapter — and that would not change the fact that our two dimensions are bridged.

References

- E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 — Bridge (pp. 151–161), Adapter (pp. 139–150).
- Refactoring Guru. “Bridge” and “Adapter” design patterns. https://refactoring.guru/design-patterns (accessed 2026).
- Lecture 4 “Bridge Pattern” (ShP-2216) and the Bridge reading on Moodle. Astana IT University, 2026–2027.
- Oracle. *Java SE 17 API Documentation*. https://docs.oracle.com/en/java/javase/17/.